

Kubernetes (K8s) Infrastructure Connectivity for the Modern Data Center

cisco Live !

Camillo Rossi
Technical Leader

Webex App

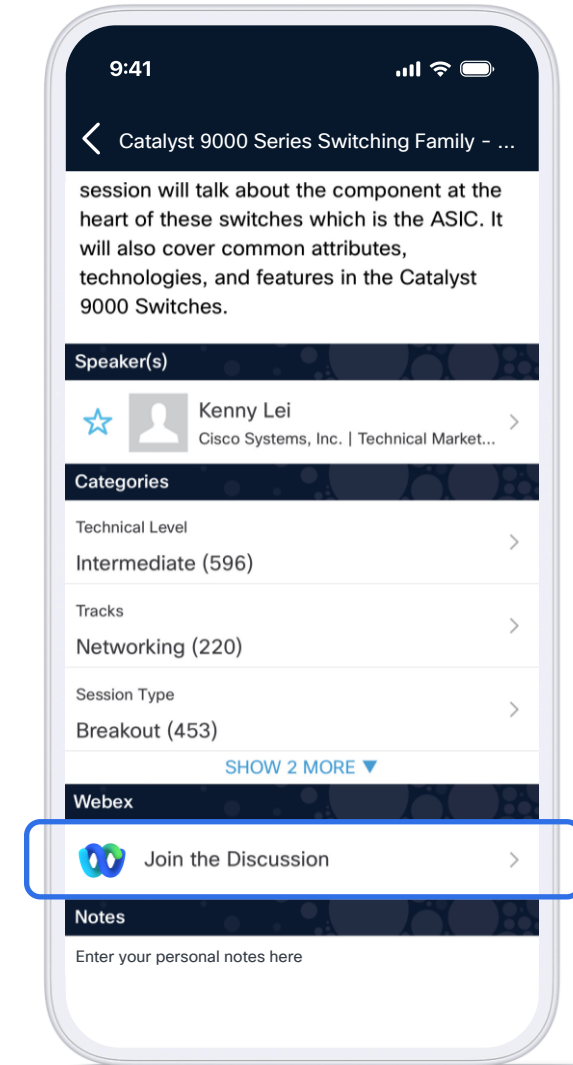
Questions?

Use Webex App to chat with the speaker after the session

How

- 1 Find this session in the Cisco Events App
- 2 Click “Join the Discussion”
- 3 Install the Webex App or go directly to the Webex space
- 4 Enter messages/questions in the Webex space

Webex spaces will be moderated by the speaker until February 27, 2026.



Agenda

- 01 **Kubernetes Refresh**
- 02 **Network Challenges**
- 03 **BGP Based Architecture**
- 04 **Cilium**
- 05 **ACI Design**
- 06 **NX-OS Design**
- 07 **Demos**
- 08 **Q&A**

Kubernetes Refresh

What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform. It is designed to automate the deployment, scaling, and management of containerized applications. Originally developed by Google, Kubernetes is now maintained by the Cloud Native Computing Foundation (CNCF).

1. Container Orchestration:

- Manages the lifecycle of containers across a cluster of machines.
- Ensures containers are deployed, running, and healthy.

2. Self-Healing:

- Automatically restarts containers that fail, replaces unhealthy nodes, and reschedules workloads as needed.

3. Scalability:

- Supports horizontal scaling (adding/removing replicas of applications) based on traffic or resource demands.

4. Load Balancing:

- Distributes traffic across multiple containers to ensure high availability and performance.

5. Declarative Configuration:

- Users describe the desired state of the system (e.g., "I want 5 replicas of this app"), and Kubernetes ensures the actual state matches the desired state.

6. Portability:

- Works across cloud providers, on-premises data centers, and hybrid environments.

Key Kubernetes Concepts

- **Pods:**

- Smallest deployable unit.
- Encapsulates one or more containers with shared storage/network.

- **Deployments:**

- Declarative updates for Pods.
- Automatic Rollouts and Rollbacks
- Scaling: Allows you to define how many instances of your app should run

- **Services:**

- Expose your application to external/internal traffic.
- Types: ClusterIP, NodePort, LoadBalancer, etc.

- **ConfigMaps & Secrets:**

- Manage configuration data.
- Secrets securely store sensitive information.

- **Namespaces:**

- Logical isolation within a cluster.
- Useful for separating environments (e.g., dev, prod)

Network Challenges

Network Challenges – In General

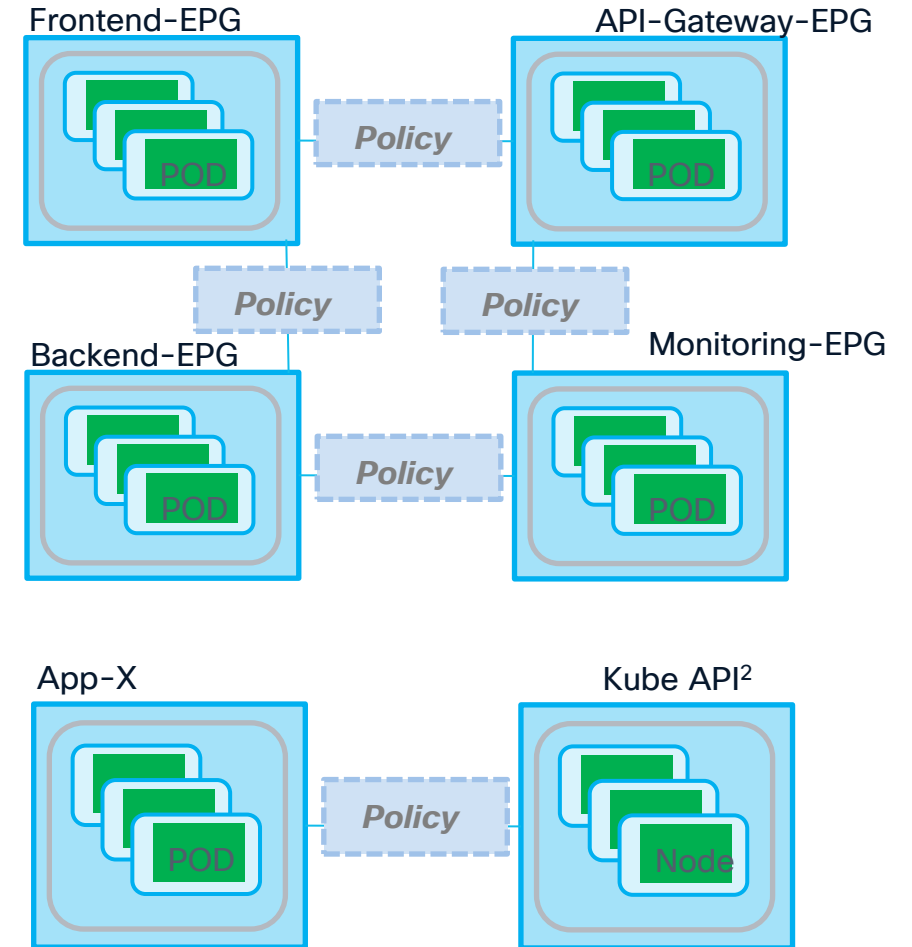
- Kubernetes networking involves different layers and components, such as pods, services, ingress, network policies, and more. Each component has its own networking requirements, making it complex to manage and understand.
- Kubernetes itself does not provide built-in networking functionality. Instead, it relies on the CNI specification to allow third-party plugins to handle the networking. This approach makes Kubernetes modular and flexible, enabling users to choose the best networking solution for their specific needs however has also led to the proliferation of several CNI Plugins all with different capabilities.
- Navigating the maze of Kubernetes CNI options can be challenging, as each plugin offers unique features and trade-offs that must align with your specific networking and security needs.



Different Traffic Flows: Different Needs

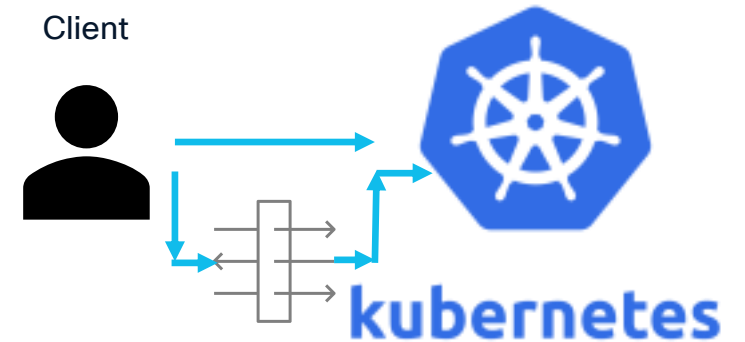
East – West

- These are communication flows inside the cluster:
 - Pod-to-Pod
 - Pod-to-Node
- This traffic is automatically handled by the CNI plugin of choice
- Kubernetes Network Policies can be used to secure the traffic in a Cloud native way¹
- **Main Challenges:**
 - **Visibility:** The flows are invisible to the fabric due to overlays making troubleshooting hard especially if Kubernetes Network Policies are used
- **Solutions:**
 - Use a CNI that provides in depth visibility like Isovalent and Timescape
 - Remove the Overlay to allow the fabric to see the traffic flows



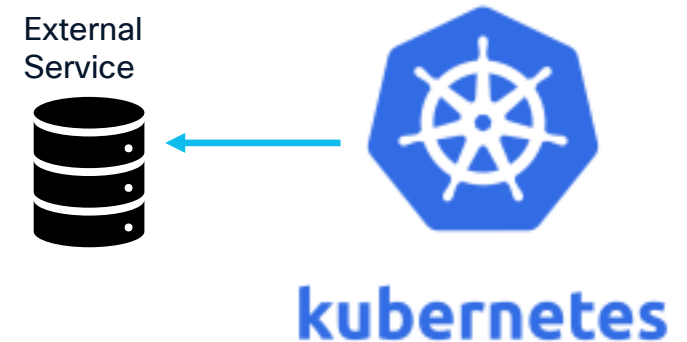
North -> South (ingress)

- These communication flows originate from entities outside the cluster that consume its services.
- Exposing services to external users or systems is complex
- **Main Challenges:**
 - **Service Advertisement:** How are the Services Advertised to the network?
 - **LoadBalancing :** How is the traffic Load Balanced effectively between the K8s Nodes hosting the PODs?
 - **Security:** How do we insert a Perimeter Firewall?¹
- **Solutions:**
 - Integrate the CNI with the network



South -> North (egress)

- These communication flows are initiated by entities within the cluster (Pods) that consume external services. (Pod-Initiated Traffic)
- By default, POD initiated traffic is NATed to the Node IP
- In most cases using only K8s Network policies for security is not sufficient
- **Main Challenges:**
 - **Security:** Securing pod-initiated traffic to an external systems is difficult:
 - If the Pod IP is NATed to the Node IP, the best identifiable source becomes "the cluster" as individual Pod-level identity is lost in the process.
 - If the Pod IP is routable (advertised to the fabric), its ephemeral nature poses significant challenges for configuring external firewalls.
 - How do we insert a Perimeter Firewall?
- **Solutions:**
 - Map the egress flows to a predictable IP

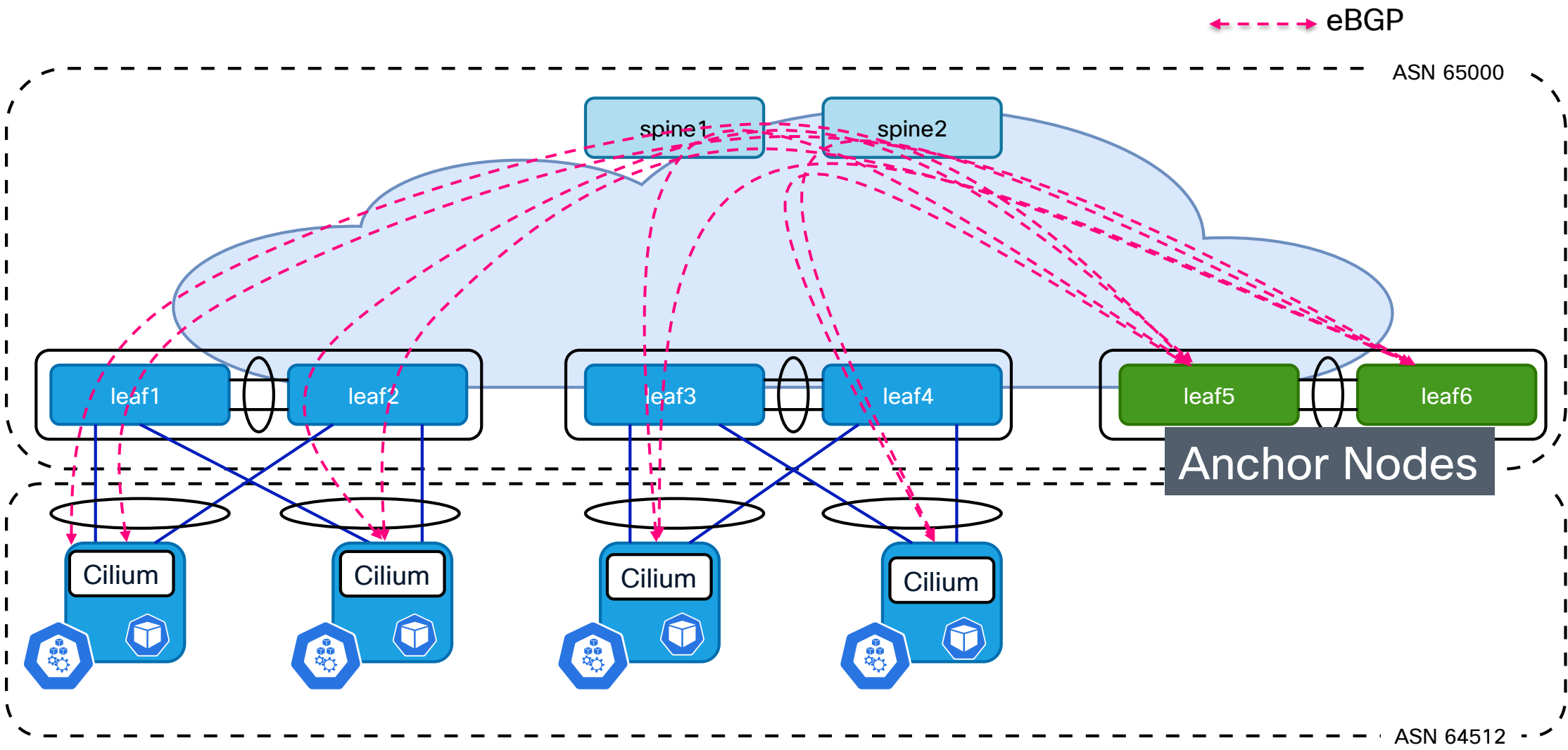


BGP Based Architecture

BGP Based Integration

- Identical Design for ACI and NX-OS VXLAN EVPN
- Leverages a Proven Protocol: Built on BGP for reliable and scalable routing.
- Unified L3 Networking:
 - Seamless Layer 3 connectivity for Kubernetes Nodes, Pods, and Service endpoints.
 - Enables effortless communication both within and outside the fabric.
- Enhanced Security (ACI/NX-OS):
 - Security group classification to secure traffic across Node, Pod, and Service subnets.
 - Service Insertion for external Firewall via Service Graphs or PBR Redirection
- Optimized Performance:
 - High-performance, low-latency connectivity without the overhead of overlays.
- Scalable Load Balancing:
 - Hardware-assisted ECMP (Equal-Cost Multi-Path) routing, ensuring efficient traffic distribution and scalability.
- Flexible Deployment Options:
 - Compatible with Any Hypervisor or Bare Metal, enabling a mix of deployment form factors for maximum flexibility.

Centralized Routing Peering



BGP Tuning

- BGP Dynamic Neighbours
- BGP Graceful Restart
- BFD (Requires Isovalent Enterprise)
- Increase Max BGP ECMP paths for better load balancing

BGP Hardening

- Enabled BGP password authentication
- Set the maximum AS limit to one
- Configure BGP import route control to accept only the expected subnets from the Kubernetes cluster:
 - Pod subnet(s)
 - Node subnet(s)
 - Service subnet(s)
- Set a limit on the number of received prefixes from the nodes.

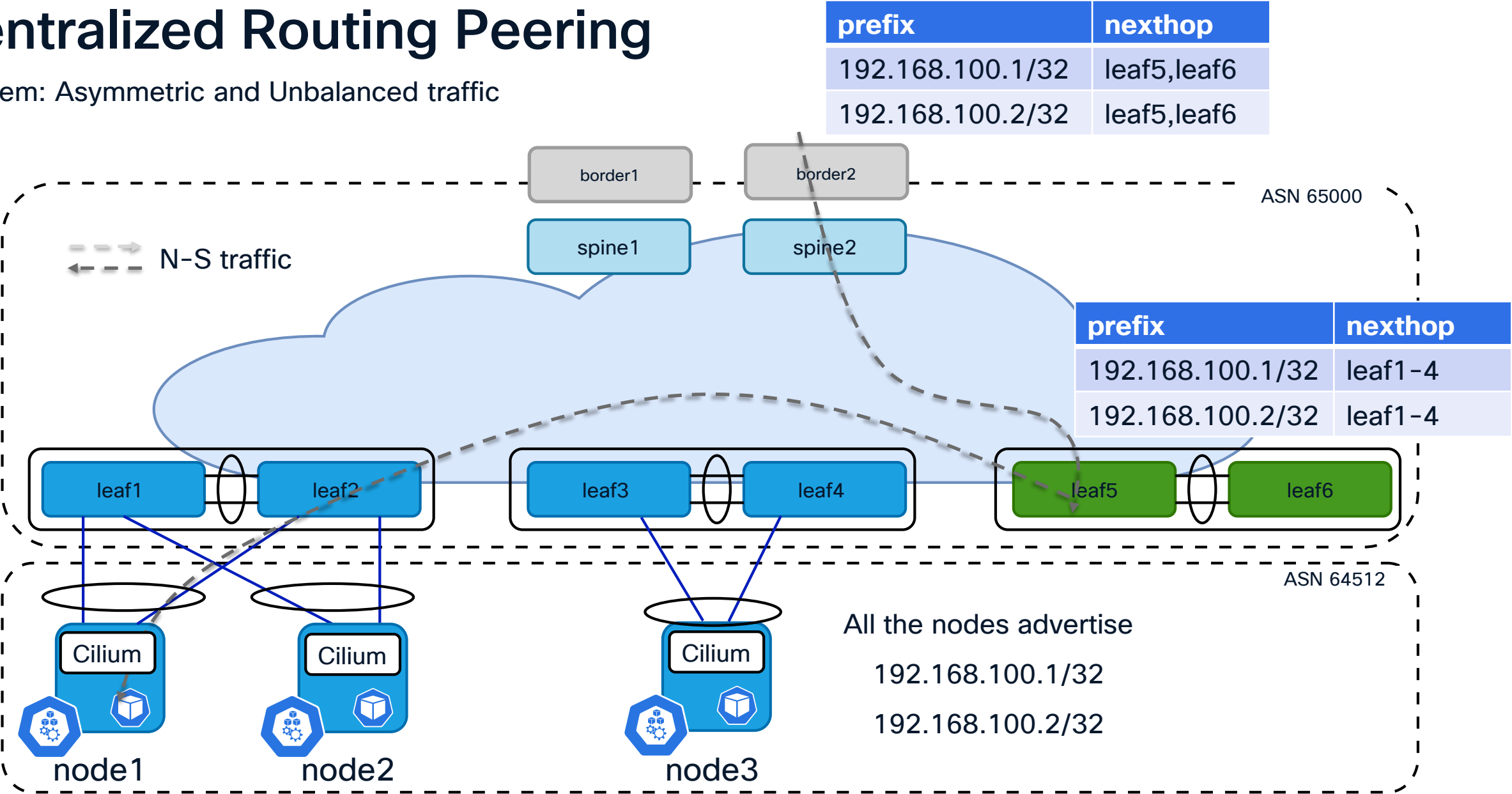
Expected Routing Behavior

- Nodes service IPs (and optionally pods)Subnets will be advertised to the fabric
- Exposed Services will be advertised as host routes from every nodes that has a running POD associated to the service.
- Optimal Routing in place with ACI 6.1(3), NX-OS 10.5 or later

Traffic Flow Optimizations

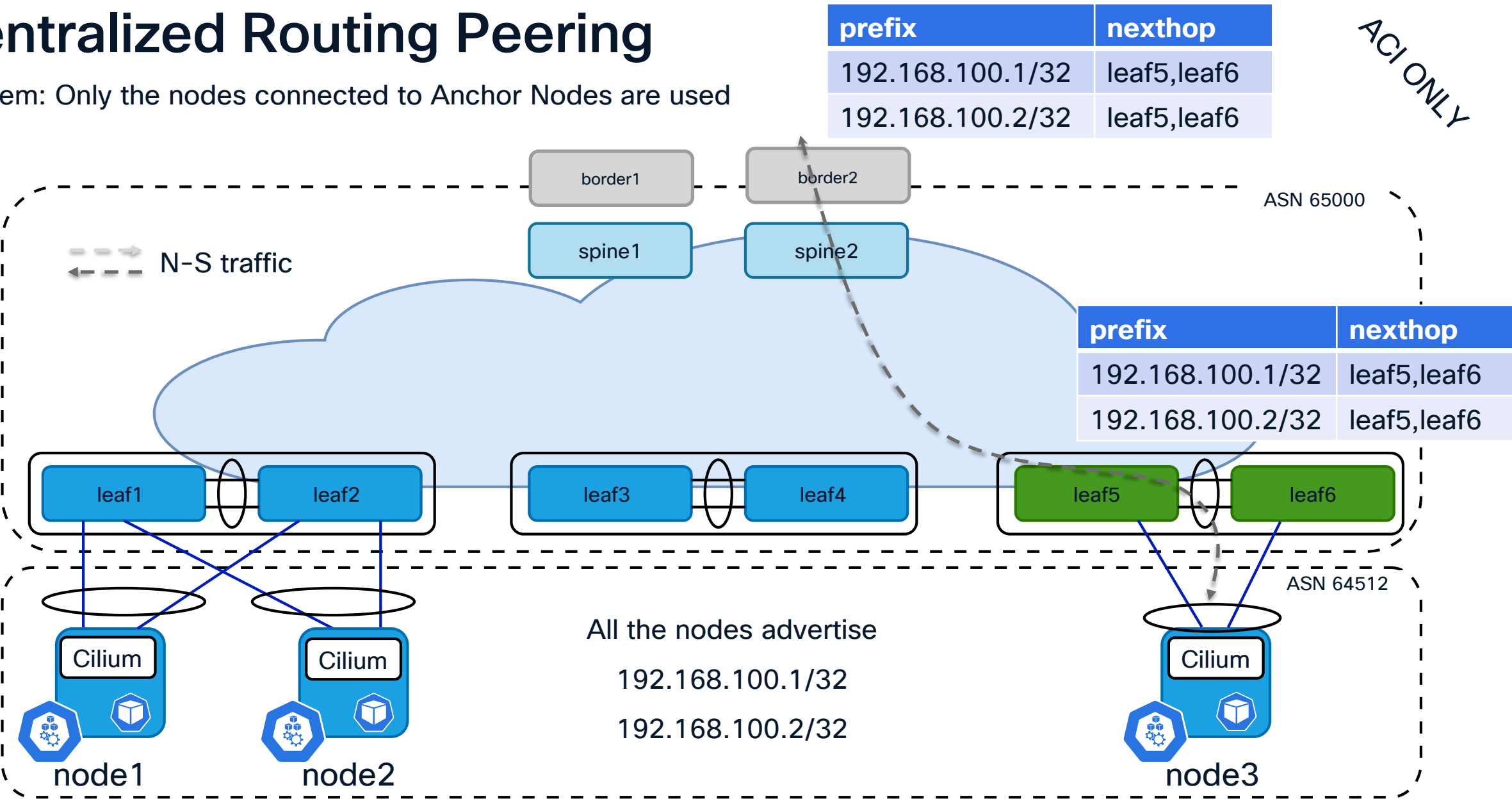
Centralized Routing Peering

Problem: Asymmetric and Unbalanced traffic



Centralized Routing Peering

Problem: Only the nodes connected to Anchor Nodes are used



Centralized Routing Peering

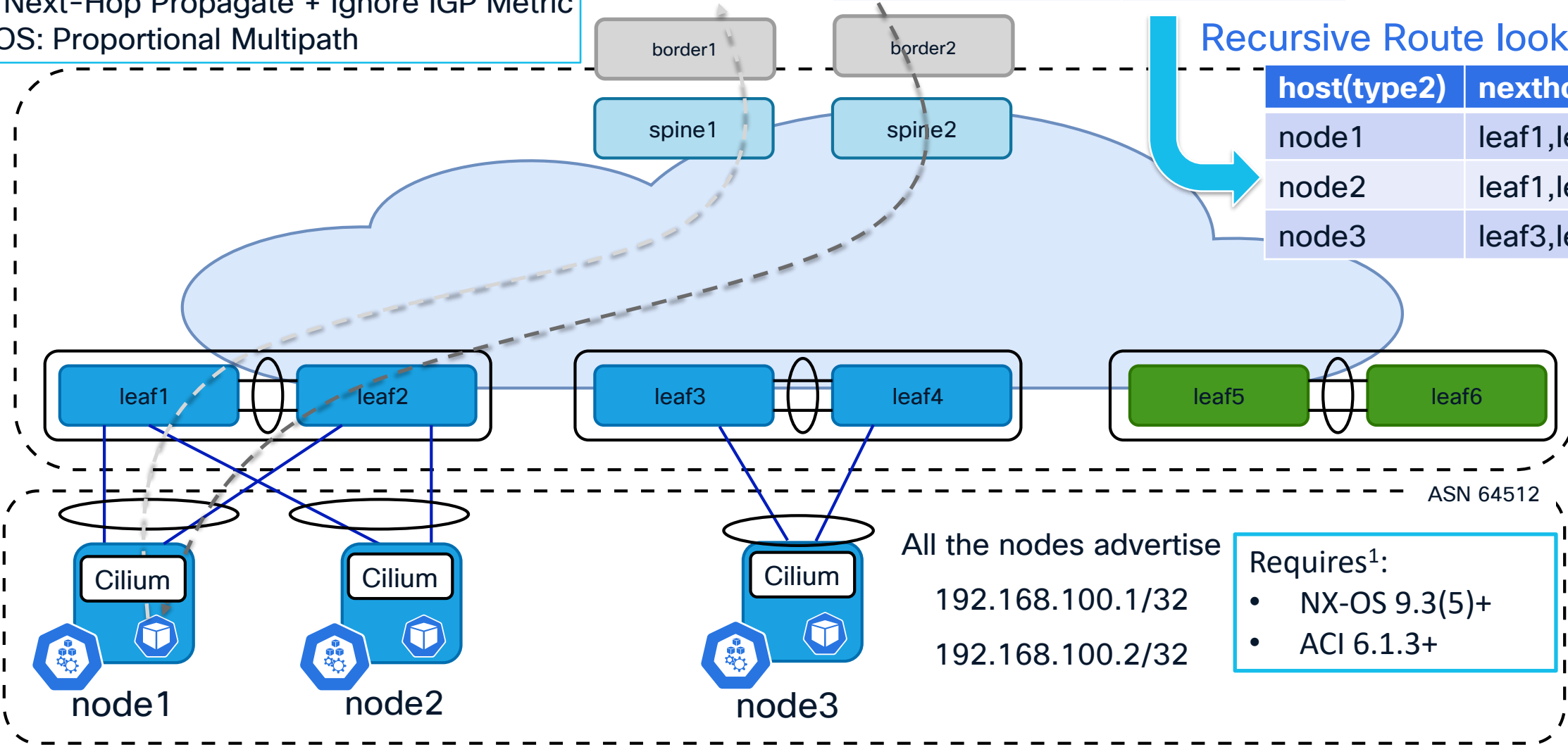
Solution

ACI: Next-Hop Propagate + Ignore IGP Metric
NX-OS: Proportional Multipath

prefix	nexthop
192.168.100.1/32	node1-3
192.168.100.2/32	node1-3

Recursive Route lookup

host(type2)	nexthop
node1	leaf1,leaf2
node2	leaf1,leaf2
node3	leaf3,leaf4

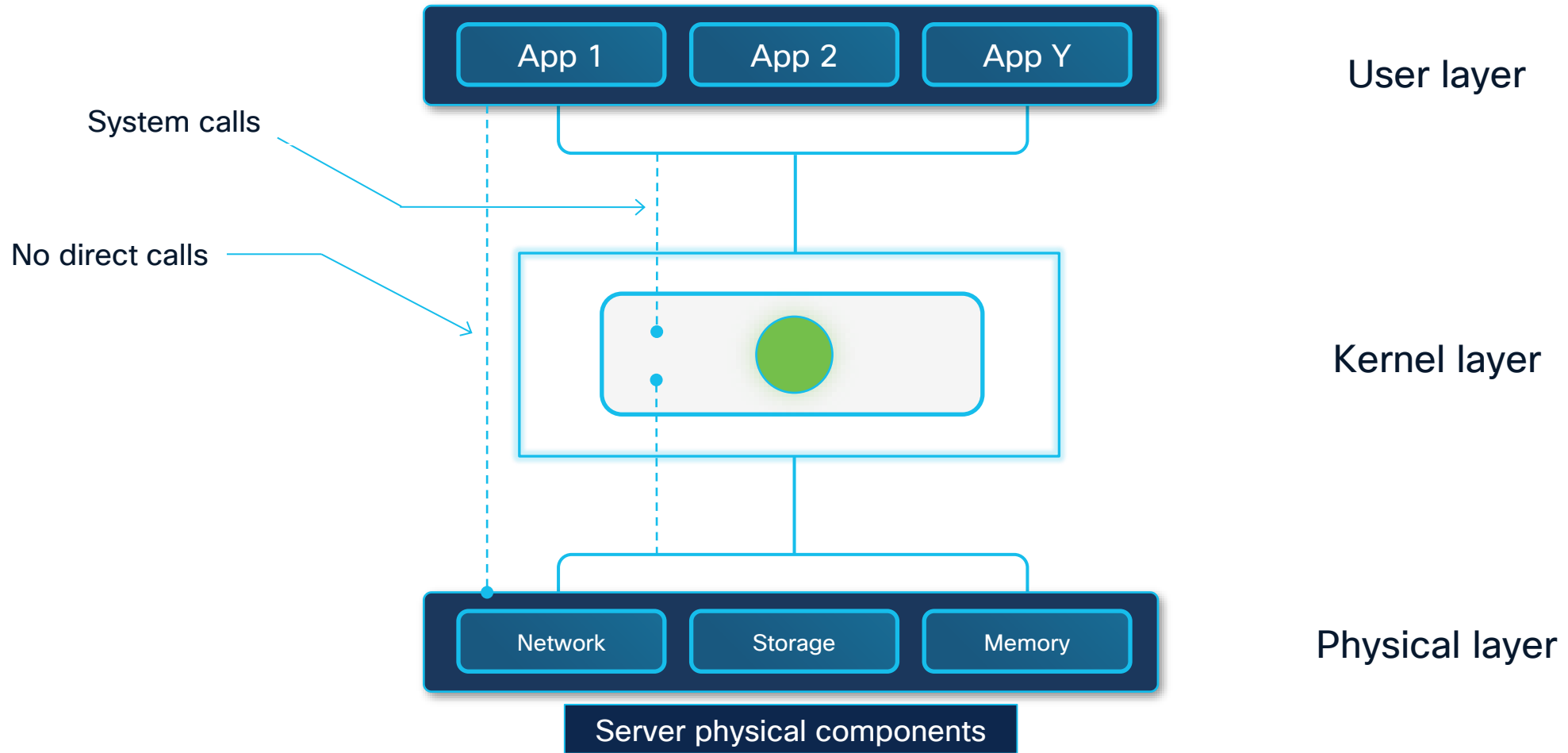


Isovalent Networking for Kubernetes (Cilium)

What is it?

- Cilium is an open source, cloud native solution for providing, securing, and observing network connectivity between workloads.
- Cilium was created by Isovalent (joined Cisco via acquisition in April 2024.) and is currently the only CNCF graduated networking plugin project.
- The enterprise version is called **Isovalent Networking for Kubernetes**
- **Advanced Network Security:** fine-grained security policies that can be defined based on application-layer identities rather than just IP addresses. This allows for more precise control over communication between microservices.
- **Scalability and Performance:** By using eBPF it achieves high performance and scalability, as it can run network policies directly in the Linux kernel without the need for complex user-space processing.
- **Service Mesh Integration:** integrated with service mesh architectures to provide transparent, high-performance networking and security features that complement service mesh capabilities.
- **Observability:** Robust observability tools, allowing users to monitor and troubleshoot network traffic and security policies with detailed metrics and visibility into application communication patterns.
- **Kubernetes Native:** Designed with Kubernetes in mind seamlessly integrates with Kubernetes to provide native support for network policies and service discovery

How do operating systems work?



eBPF – Foundation of Cilium

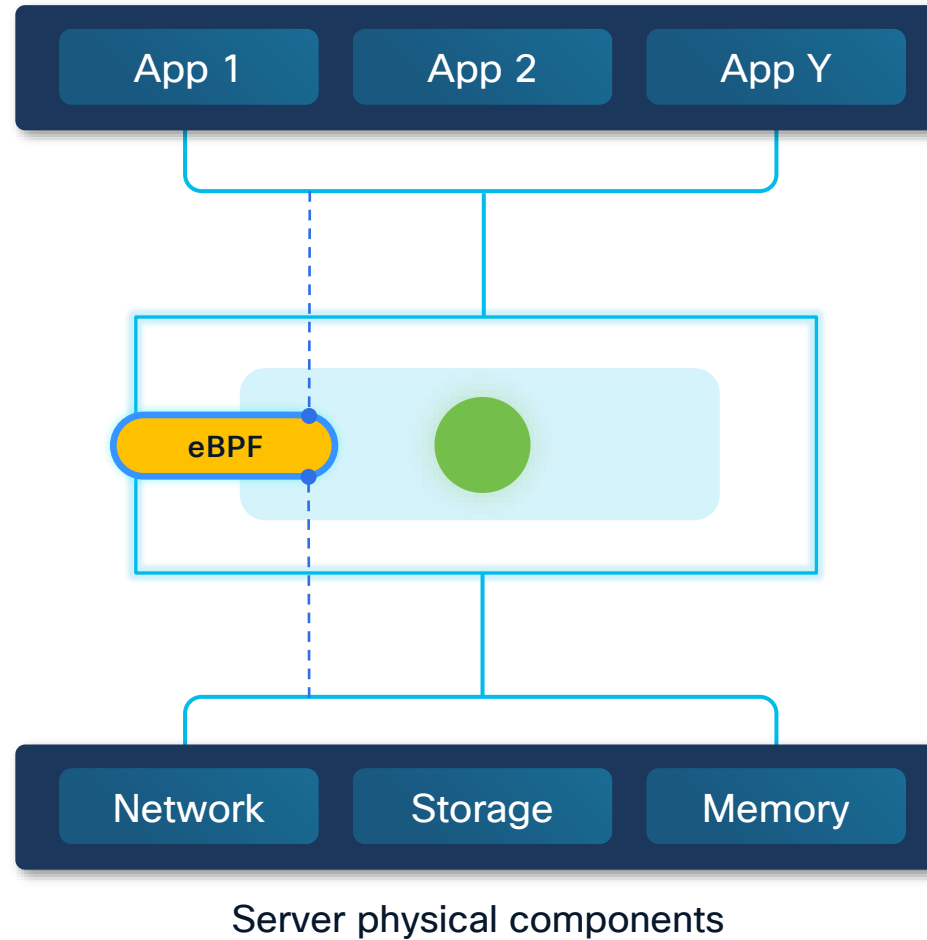


Network filtering

Observability

Security policy

What Javascript is
to the browser,
eBPF is to the kernel.



User layer

Kernel layer

Physical layer

eBPF – Foundation of Cilium

- Kubernetes networking
- Load balancing
- Kubernetes services
- Identity-based security
- L7 policies

- Dependencies map (service and flows)
- Monitoring and alerting
- App monitoring

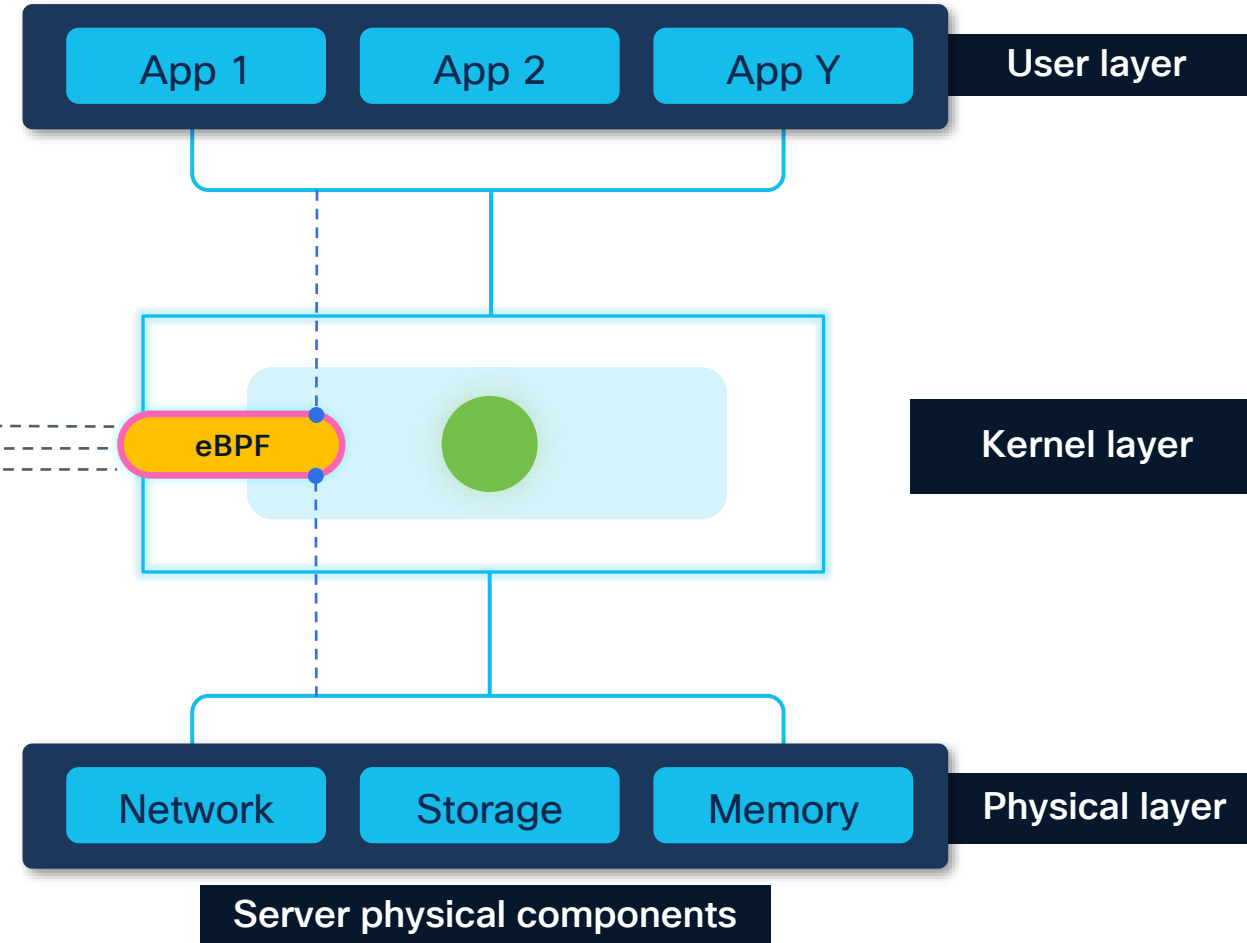
- Monitor process execution
- Runtime security policies
- Real time enforcement



Network filtering

Observability

Security policy

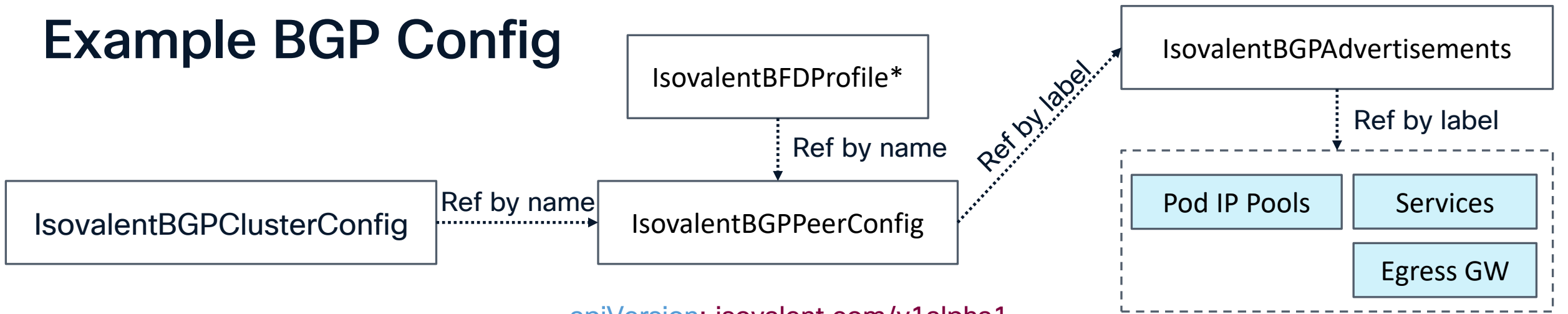


How to BGP with Cilium?

We only need 4 configs

- **IsovalentBGPClusterConfig**: Defines BGP instances and peer configurations that are applied to multiple nodes.
- **IsovalentBGPPeerConfig**: A common set of BGP peering setting. It can be used across multiple peers.
- **IsovalentBGPAdvertisements**: Defines prefixes that are injected into the BGP routing table.
- **IsovalentBFDProfile**: Defines the BFD parameters

Example BGP Config



```
apiVersion: isovalent.com/v1alpha1
kind: IsovalentBGPClusterConfig
metadata:
  name: cilium-bgp
spec:
  bgpInstances:
    - name: "instance-65113"
  localASN: 65113
  peers:
    - name: "leaf201"
      peerASN: 65002
      peerAddress: 192.168.2.43
      peerConfigRef:
        name: "cilium-peer"
```

```
apiVersion: isovalent.com/v1alpha1
kind: IsovalentBGPPeerConfig
metadata:
  name: cilium-peer
spec:
  ebgpMultihop: 1
  gracefulRestart:
    enabled: true
  families:
    - afi: ipv4
      safi: unicast
      advertisements:
        matchLabels:
          advertise: "bgp"
  bfdProfileRef: tor-bfd-profile
```

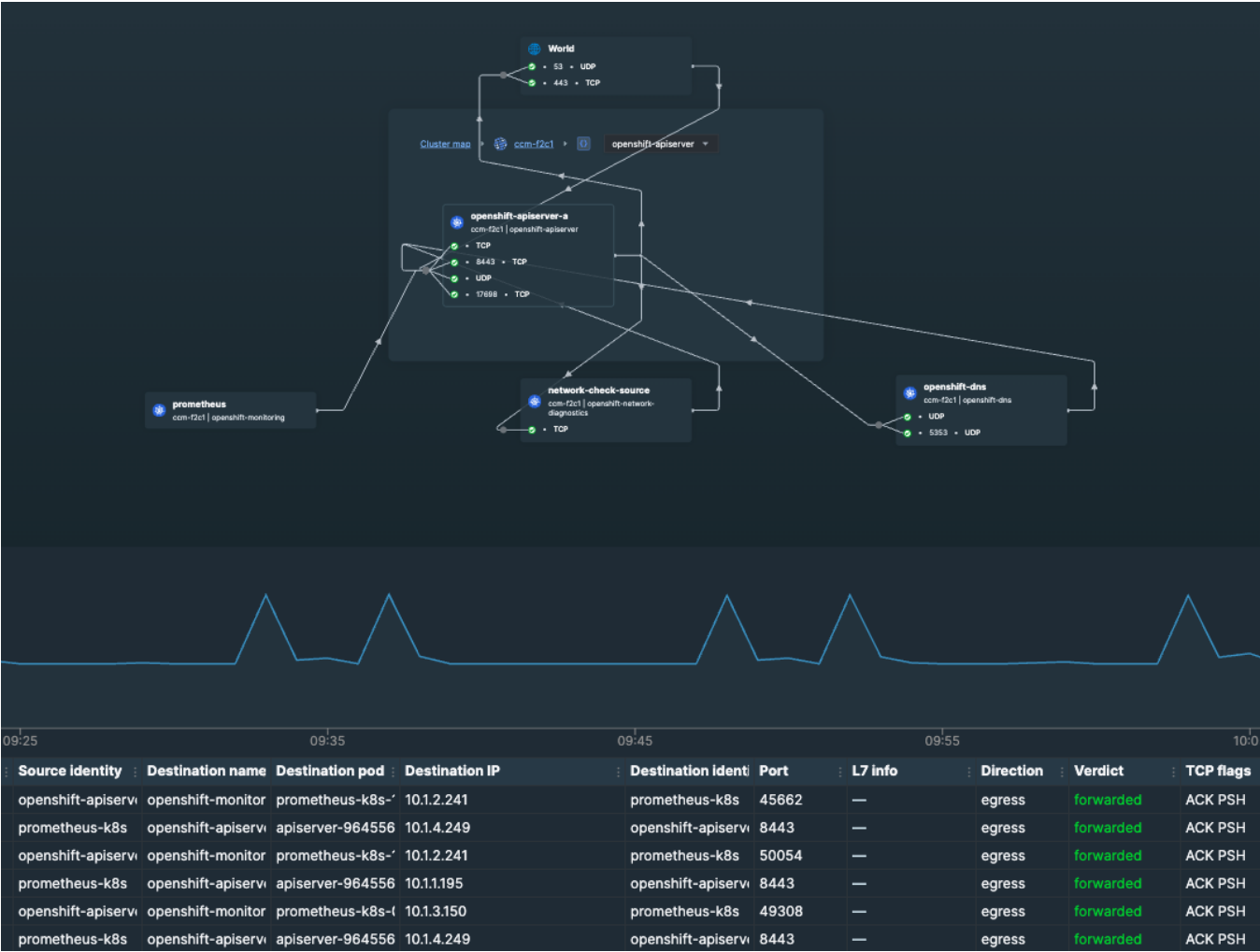
```
apiVersion: isovalent.com/v1alpha1
kind: IsovalentBGPAdvertisement
metadata:
  name: bgp-advertisements
labels:
  advertise: bgp
spec:
  advertisements:
    - advertisementType: "Service"
      service:
        addresses:
          - LoadBalancerIP
      selector:
        matchLabels:
          advertise: bgp
```


Addressing the challenges

East – West Visibility

Timescape

- Isovalent Timescape is an observability and forensics platform that provides historical visibility and long-term storage for network flows, security events, and policy changes within Kubernetes environments.
- It is part of the Isovalent Enterprise Platform



Demo - Visibility

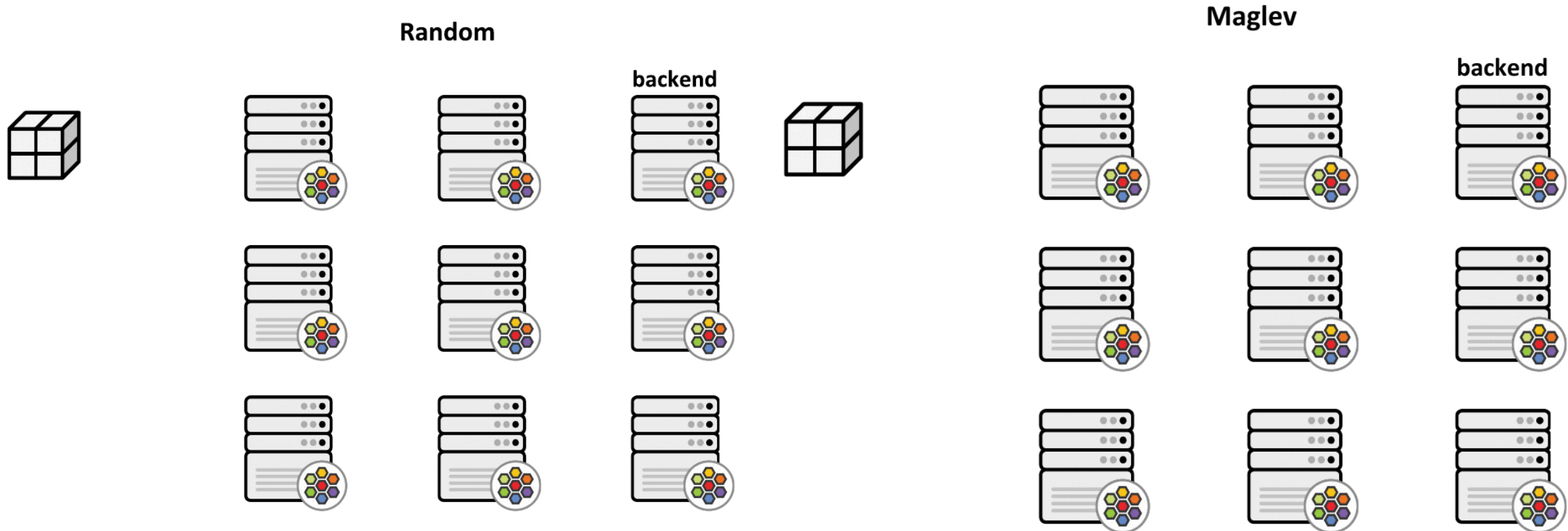
North -> South

Running BGP!

- By running BGP we are addressing all the challenges we had:
 - The services are advertised, load balanced with ECMP and inserting a firewall become a trivial task.
- Bonus: Use ACI ExtEPG or NXOS GPO for Fabric Security
- **However, there is one issue we didn't discuss yet...**

ECMP Resilient Hashing or the lack of it...

- ECMP resilient hashing is not available on all platforms, and this can result in TCP flows reset.
- Maglev consistent hashing minimizes disruptions by ensuring all load balancing nodes share a consistent backend lookup table. Using the packet's 5-tuple hash, traffic is reliably forwarded to the same backend without state synchronization. This enhances resiliency during failures and ensures effective load balancing when new nodes are added.

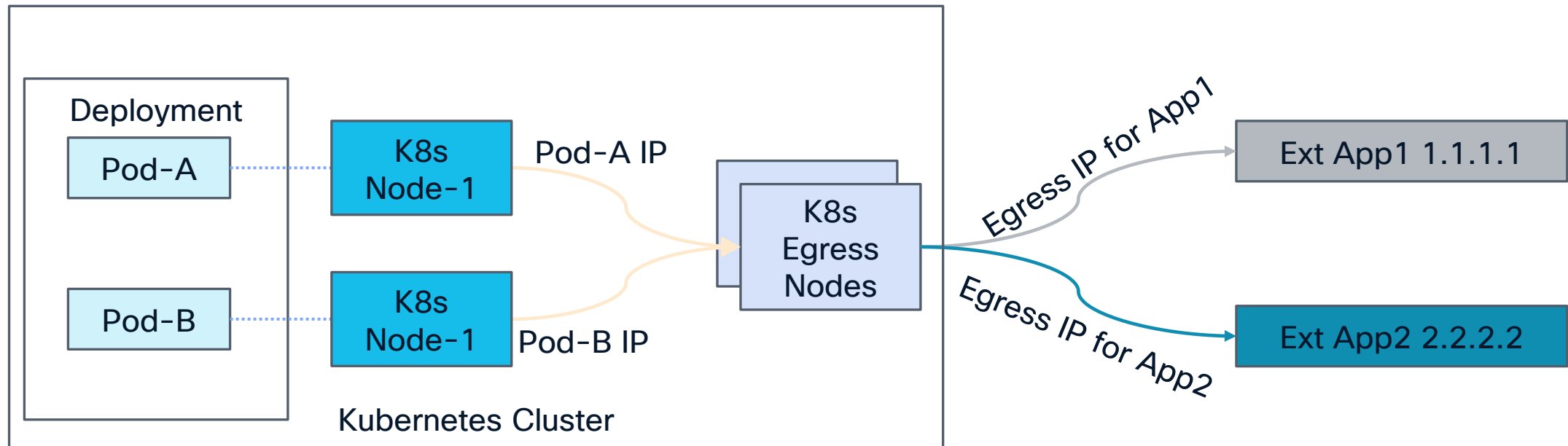


Demo – BGP and Services

South -> North

Isovalent Egress Gateway

- The Egress Gateway features allows for redirecting traffic originating from pods and destined to specific CIDRs outside the cluster to be routed through particular nodes.
- When the egress gateway policies are in place, packets leaving the cluster are masqueraded with selected, predictable IPs (egressIP) associated with the gateway nodes. This allows for network administrators to apply network controls to pods and or namespaces establishing outbound connectivity.



Demo – Egress Gateways

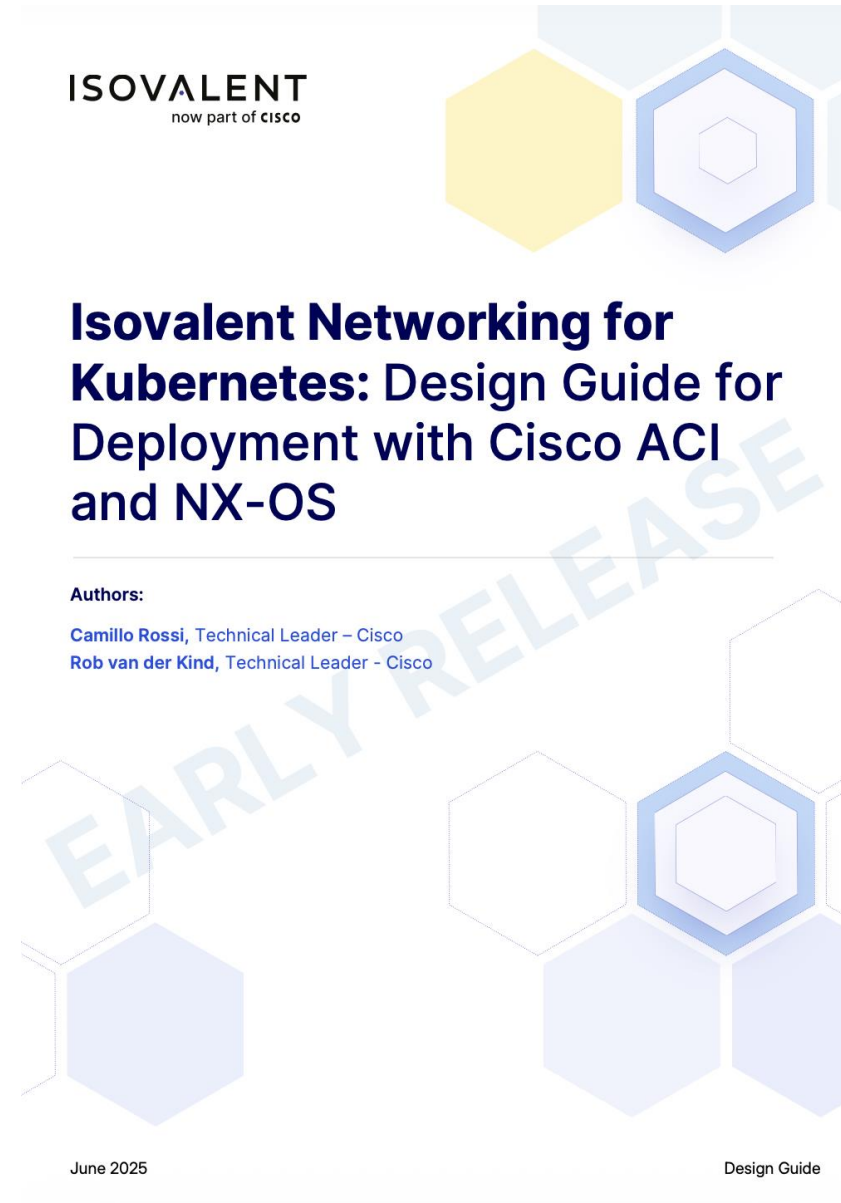
Conclusion

Isovalent and Cisco DC Fabric – Better Together

- **Easy and Scalable:**
 - BGP is a well establish protocol easy to configure and troubleshoot
 - High Scale with thousand nodes per cluster
- **Visibility:**
 - Every communication flow can be monitored with Timescape
 - Traffic Traversing the network can be monitored with conventional tools
- **Security:**
 - Could Native Security enforced by Isovalent using eBPF
 - Network level security (Node to Node, N/S or S/N) leveraging fabric contracts or service insertion
- **NetOps Ready:**
 - Kubernetes ACI and NX-OS can be driven by API, ansible or terraform ensuring automation is a breeze

Get the design guide!

- Free Download at: <https://isovalent.com/briefs/designing-isovalent-enterprise-for-cisco-aci-and-nexus/>



Complete your session surveys



Complete your surveys in the Cisco Events App.



Complete a minimum of 4 session surveys and the overall event survey to receive a unique Cisco Live t-shirt.
(from 11:30 on Thursday, while supplies last)

Continue your education



Visit the Cisco Showcase for related demos



Book your one-on-one Meet the Engineer meeting

Visit the Technical Solutions Clinics to discuss your technical questions



Attend the interactive education with DevNet, Capture the Flag, and Walk-in Labs



Visit the On-Demand Library for more sessions at CiscoLive.com/On-Demand

Thank you

CISCO Live !

